TDP — Tradutor de Palavras

# PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

## Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.0

| Campo | Descrição |
|---|---|
| Projeto | TDP — Tradutor de Palavras |
| Gerente de Projeto | João Filipi Girardi Tissi |
| Gerente/Responsável de GCS-DevOps | João Filipi Girardi Tissi |
| Fábrica de Software / Squad | Individual — João Filipi Girardi Tissi |
| Repositório principal | https://github.com/joaofilipigtissi/TradutorDePalavras.git |

### Histórico de Revisões

| Data | Versão | Descrição | Autor |
|---|---|---|---|
| 23/09/2026 | 1.0 | Elaboração do documento com enfoque em práticas DevOps (CI/CD), adaptado ao projeto individual Tradutor de Palavras. | João Filipi Girardi Tissi |
| | | | |

# Sumário

**Sumário** ..... 2
**1. Introdução** ..... 4
**1.1 Finalidade** ..... 4
**1.2 Escopo** ..... 4
**1.3 Definições, Acrônimos e Abreviações** ..... 4
**1.4 Referências** ..... 4
**1.5 Visão Geral** ..... 5
**2. Cultura DevOps e Papéis na Gerência de Configuração** ..... 5
**3. Ferramentas, Ambientes e Infraestrutura** ..... 5
**3.1 Ferramentas do Pipeline** ..... 5
**3.2 Ambientes e Infraestrutura** ..... 6
**3.3 Estratégia de Branching e Versionamento** ..... 6
**4. Identificação da Configuração** ..... 7
**4.1 Itens de Configuração** ..... 7
**4.2 Convenção de Nomenclatura** ..... 7
**4.3 Estrutura do Repositório** ..... 7
**5. Integração Contínua (CI)** ..... 8
**5.1 Etapas do Pipeline de CI** ..... 8
**5.2 Política de Pull Request e Revisão de Código** ..... 8
**5.3 Critérios de Qualidade (Quality Gate)** ..... 8
**6. Entrega e Implantação Contínua (CD)** ..... 8
**6.1 Etapas do Pipeline de CD** ..... 8
**6.2 Estratégias de Implantação** ..... 8
**6.3 Rollback e Recuperação** ..... 9
**7. Infraestrutura como Código (IaC)** ..... 9
**7.1 Versionamento de Infraestrutura** ..... 9
**7.2 Ambientes Efêmeros** ..... 9
**8. Controle de Configuração e Mudança** ..... 9
**8.1 Processo de Solicitação de Mudança** ..... 9
**8.2 Comitê de Controle de Mudança (CCM)** ..... 10
**9. Baselines e Releases** ..... 10
**9.1 Geração Automática de Baseline** ..... 10
**9.2 Notas de Versão (Release Notes)** ..... 10
**10. Estimativa do Status de Configuração** ..... 10
**10.1 Monitoramento e Observabilidade** ..... 10
**10.2 Métricas DORA** ..... 10
**10.3 Relatórios e Auditorias** ..... 11
**11. Marcos** ..... 11

**12. Treinamento e Recursos** ..... 11
**13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros** ..... 11
**14. Quadro-Resumo do Pipeline CI/CD** ..... 11

# 1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

## 1.1 Finalidade

Estabelecer um padrão a ser seguido pela equipe/individuo, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

## 1.2 Escopo

*Este PGC é destinado ao estudante João Filipi Girardi Tissi, responsável individualmente pelo desenvolvimento, versionamento e evolução do aplicativo Tradutor de Palavras — um aplicativo Flutter de tela única, desenvolvido em Dart, que permite ao usuário visualizar uma palavra em Português e suas traduções para Inglês e Francês, navegando entre uma lista de palavras predefinida. O plano cobre o código-fonte da aplicação, hospedado no repositório GitHub do projeto, e as práticas de controle de versão, integração e futura automação de build/análise adotadas ao longo do ciclo de vida do projeto.*

## 1.3 Definições, Acrônimos e Abreviações

| Termo | Significado |
|---|---|
| GCS | Gestão de Configuração de Software |
| CI | Integração Contínua (Continuous Integration) |
| CD | Entrega/Implantação Contínua (Continuous Delivery / Deployment) |
| IaC | Infraestrutura como Código (Infrastructure as Code) |
| CCM | Comitê de Controle de Mudanças |
| Baseline | Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade |
| Pipeline | Sequência automatizada de etapas de build, teste, análise e implantação |
| Trunk-based Development | Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência |
| SemVer | Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0. |
| DORA Metrics | Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate) |

## 1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.

*Repositório do projeto: https://github.com/joaofilipigtissi/TradutorDePalavras.git. Não há, neste momento, Plano de Projeto, Cronograma ou Documento de Arquitetura formais, dado o caráter individual e acadêmico do projeto.*

### 1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: cada commit integrado, testado e implantado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados), reduzindo o esforço manual e o tempo entre a mudança e sua entrega em produção.

## 2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto **DevOps é responsabilidade compartilhada entre desenvolvimento e operações ("Você constrói, você executa" — you build it, you run it), apoiada por automação.** Os papéis abaixo descrevem responsabilidades típicas, que podem ser exercidas por uma mesma pessoa em squads pequenas.

Por se tratar de um projeto individual, todos os papéis descritos na tabela abaixo são acumulados por João Filipi Girardi Tissi, que atua simultaneamente como Gerente de Projeto, Engenheiro DevOps, Desenvolvedor e responsável por QA, segurança e eventual controle de mudanças.

| Papel | Responsabilidades |
|---|---|
| Gerente de Projeto / Product Owner | Prioriza o backlog, autoriza mudanças de maior impacto e acompanha indicadores de entrega (DORA Metrics). |
| Engenheiro DevOps / Release Engineer | Projeta e mantém os pipelines de CI/CD, define a estratégia de *branching* e versionamento, administra o registro de artefatos/imagens e a IaC. |
| Desenvolvedor(a) | Integra código com frequência na branch principal, escreve testes automatizados, revisa Pull Requests e é corresponsável pela qualidade do que é implantado. |
| SRE / Operações *Site Reliability Engineering* | Monitora ambientes de produção, define SLIs/SLOs, participa de postmortems e realiza rollback quando necessário. - *Service Level Indicators (SLIs) and Service Level Objectives (SLOs).* |
| QA / Automação de Testes | Mantém a suíte de testes automatizados (unitários, integração, aceitação) executada nos pipelines de CI e CD. |
| Segurança (DevSecOps) | Define políticas de análise estática (SAST), análise de dependências (SCA) e gestão de segredos integradas ao pipeline. |
| Comitê de Controle de Mudanças (CCM) | Atua apenas em mudanças de alto risco/impacto (ex.: mudanças em produção fora do fluxo padrão); mudanças de rotina seguem o fluxo automatizado de PR + pipeline. |

## 3. Ferramentas, Ambientes e Infraestrutura

### 3.1 Ferramentas do Pipeline

Ferramentas efetivamente adotadas (ou planejadas) pelo projeto, dado o seu escopo individual e acadêmico:

| Categoria | Ferramenta(s) sugerida(s) | Finalidade |
|---|---|---|
| Controle de versão | Git + GitHub (adotado) | Versionamento de código-fonte, IaC e pipelines como (build + testes verdes) código |
| Orquestração de CI/CD | GitHub Actions (planejado) | Automação de build, testes, análise e implantação |
| Empacotamento | Não aplicável — build nativo do Flutter (flutter build web / apk) | Geração de imagens/artefatos versionados e imutáveis |
| Registro de artefatos | GitHub Releases (planejado, para publicar builds/APKs) | Armazenamento e versionamento de imagens e pacotes |
| Infraestrutura como Código | Não aplicável — aplicativo cliente, sem infraestrutura de servidor própria | Provisionamento e configuração versionada de ambientes |
| Orquestração de containers | Não aplicável nesta fase do projeto | Execução e escalonamento dos serviços implantados |
| Gestão de segredos | GitHub Secrets (planejado; não há segredos no projeto atual) | Armazenamento seguro de credenciais usadas nos pipelines |
| Monitoramento e Observabilidade | Não aplicável nesta fase (ver seção 10) | Métricas, logs e alertas dos ambientes |
| Gestão de dependências / SCA – *Software Component Analisys* | Dependabot (planejado — nativo do GitHub) | Identificação de vulnerabilidades em bibliotecas de terceiros |
| Rastreamento de tarefas | GitHub Issues (adotado) | Vinculação de commits/PRs a itens de trabalho |

## 3.2 Ambientes e Infraestrutura

Os ambientes devem ser, sempre que possível, provisionados via Infraestrutura como Código, garantindo paridade entre eles e permitindo recriação rápida e rastreável.

| Ambiente | Finalidade | Gatilho de promoção |
|---|---|---|
| Desenvolvimento | Execução local do aplicativo (emulador Android ou navegador Chrome) durante o desenvolvimento e testes manuais. | Execução manual pelo desenvolvedor (flutter run) |
| Homologação / Staging | Não aplicável nesta fase do projeto — dado o escopo individual e acadêmico, a validação funcional é feita diretamente pelo estudante antes do merge na branch principal. | Não aplicável |
| Produção (Release) | Versão estável do aplicativo (build web e/ou APK) publicada como Release no GitHub, disponível para avaliação e uso. | Merge na branch main + criação manual de uma tag de versão (SemVer) |

*Não aplicável: o Tradutor de Palavras é um aplicativo cliente (Flutter), sem backend, servidores, banco de dados ou domínio próprio. Não há, portanto, DNS, IPs, topologia de rede ou credenciais de acesso a detalhar nesta fase do projeto.*

## 3.3 Estratégia de Branching e Versionamento

Recomenda-se estratégia de Trunk-Based Development (ou GitHub Flow), com branches de vida curta, para viabilizar integração contínua real:

- main/trunk — sempre implantável (deployable); protegida por regras de PR obrigatório e pipeline verde.
- feature/<descrição-curta> — branches de curta duração (idealmente < 1-2 dias) para cada mudança.
- release/x.y (opcional) — utilizada apenas se houver necessidade de estabilizar uma versão antes do deploy.
- hotfix/<descrição> — correções emergenciais aplicadas diretamente sobre produção e retro-integradas ao trunk.

Versionamento: adoção de Versionamento Semântico (MAJOR.MINOR.PATCH) e Conventional Commits (feat, fix, chore, docs, refactor…), permitindo geração automática de changelog e da próxima versão pelo pipeline.

# 4. Identificação da Configuração

## 4.1 Itens de Configuração

Além do código-fonte, em um contexto DevOps os seguintes artefatos também são itens de configuração formais, versionados no mesmo repositório (ou repositórios compartilhados):

- Código-fonte da aplicação e dos testes automatizados
- Definições de pipeline como código (ex.: .github/workflows, Jenkinsfile, .gitlab-ci.yml)
- Dockerfiles e manifests de containers
- Scripts de Infraestrutura como Código (Terraform, Ansible, Helm charts, manifests Kubernetes)
- Arquivos de configuração por ambiente (variáveis, feature flags) — exceto chaves secretas, que devem ficar ambientes seguros.
- Scripts de migração de banco de dados (N/A)
- Documentação técnica (README, ADRs — Architecture Decision Records)

## 4.2 Convenção de Nomenclatura

| Item | Convenção | Exemplo |
|---|---|---|
| Branch de feature | feature/<ticket>-<descrição-curta> | feature/SIQQ-123-login-oauth |
| Commit | Conventional Commits: <tipo>(<escopo>): <descrição> | feat(auth): adiciona login via OAuth2 |
| Tag de release | Versionamento Semântico vX.Y.Z | v2.3.0 |
| Imagem de container | <registro>/<projeto>:<versão\|sha> | ghcr.io/org/projeto:2.3.0 |
| ... | ... | ... |

## 4.3 Estrutura do Repositório

*/lib — código-fonte Dart do aplicativo (main.dart); /android, /ios, /web, /windows, /linux, /macos — projetos de plataforma gerados pelo Flutter; /test — testes automatizados (a implementar); pubspec.yaml — dependências e metadados do projeto; README.md — documentação de uso e execução do projeto.*

# 5. Integração Contínua (CI)

Todo Pull Request para a branch principal deve disparar automaticamente um pipeline de CI. Nenhuma mudança é integrada sem que o pipeline seja concluído com sucesso.

## 5.1 Etapas do Pipeline de CI

1. Checkout do código e restauração de dependências (com cache).
2. Build da aplicação.
3. Análise estática de código (lint) e verificação de padrões de codificação.
4. Testes unitários, com medição de cobertura mínima.
5. Testes de integração.
6. Análise de segurança do código (SAST) e de dependências (SCA).
7. Empacotamento e publicação do artefato/imagem versionado no registro.

## 5.2 Política de Pull Request e Revisão de Código

- Todo PR exige, no mínimo, 1 aprovação de outro desenvolvedor/colega antes do merge ("quatro olhos").
- PRs devem ser pequenos e de vida curta, para reduzir conflitos e facilitar revisão.
- Merge somente permitido com pipeline de CI verde (quality gate).

## 5.3 Critérios de Qualidade (Quality Gate)

*Para este projeto: zero erros reportados por "flutter analyze"; build bem-sucedido para ao menos uma plataforma alvo (Web e/ou Android); ausência de warnings bloqueantes de compilação. Metas de cobertura de testes automatizados ficam como evolução futura do projeto (ver seção 11).*

# 6. Entrega e Implantação Contínua (CD)

Após a publicação do artefato pelo pipeline de CI, o pipeline de CD é responsável por promover essa mesma versão, sem recompilação, através dos ambientes até a produção.

Como o Tradutor de Palavras é um aplicativo cliente sem backend, o conceito de "implantação em produção" é adaptado, neste projeto, para a publicação de uma nova versão (build web e/ou APK) como Release no repositório GitHub, disponibilizando-a para uso e avaliação.

## 6.1 Etapas do Pipeline de CD

8. Deploy automático em ambiente de Homologação/Staging.
9. Execução de testes automatizados de aceitação/E2E em staging.
10. Gate de aprovação para produção (automático ou aprovação manual, conforme criticidade).
11. Deploy em produção com estratégia definida (ver 6.2).
12. Testes de fumaça (smoke tests) pós-deploy e verificação de saúde (health checks).
13. Geração automática de baseline/tag de release e notas de versão (changelog).

## 6.2 Estratégias de Implantação

| Estratégia | Descrição |
|---|---|
| Rolling update | Substituição gradual das instâncias antigas pelas novas, minimizando indisponibilidade. |
| Blue-Green | Duas versões (ambientes) coexistem; o tráfego é comutado de uma vez para a nova versão, permitindo rollback imediato. |
| Canary Release | A nova versão recebe uma pequena fração do tráfego real, sendo ampliada gradualmente conforme a confiança. |
| Feature Flags | Funcionalidades são implantadas desligadas e ativadas em produção de forma independente do deploy, reduzindo o risco de releases. |

*Nota: como o aplicativo não possui backend nem múltiplas instâncias em execução, estratégias como Rolling update, Blue-Green e Canary Release não se aplicam ao estado atual do projeto. Cada nova Release simplesmente substitui a anterior como versão recomendada para uso.*

### 6.3 Rollback e Recuperação

Toda implantação deve ser reversível. O pipeline de CD deve manter a versão anterior disponível (imagem no registro, IaC versionada) para permitir rollback automatizado em caso de falha nos testes de fumaça ou nos indicadores de saúde pós-deploy.

## 7. Infraestrutura como Código (IaC)

Toda a infraestrutura provisionada para os ambientes de Desenvolvimento, Homologação e Produção deve ser descrita como código versionado, submetida ao mesmo fluxo de Pull Request e pipeline usado para a aplicação.

*Nota: nesta fase, o projeto Tradutor de Palavras não possui infraestrutura de servidor própria (é um aplicativo cliente publicado como build web/APK), portanto as práticas de IaC descritas nesta seção são mantidas como referência para uma eventual evolução futura do projeto (ex.: caso venha a incluir um backend).*

### 7.1 Versionamento de Infraestrutura

- Alterações de infraestrutura seguem o mesmo processo de PR + revisão + pipeline ("plan" automático em PR, "apply" após aprovação).
- O estado da infraestrutura (state file) é armazenado de forma centralizada e versionada, nunca em máquinas locais.

### 7.2 Ambientes Efêmeros

Sempre que viável, ambientes de revisão (review apps) devem ser provisionados automaticamente por Pull Request, para validação isolada, e destruídos automaticamente após o merge ou fechamento do PR.

*Não aplicável nesta fase do projeto, dado que não há infraestrutura provisionável por Pull Request.*

## 8. Controle de Configuração e Mudança

### 8.1 Processo de Solicitação de Mudança

Mudanças de rotina (funcionalidades, correções) seguem o fluxo padrão: abertura de item de trabalho → branch de feature → Pull Request → revisão → pipeline de CI → merge → pipeline de CD. Esse fluxo, por si, já constitui o registro formal e rastreável da mudança.

### 8.2 Comitê de Controle de Mudança (CCM)

O CCM é acionado apenas em mudanças de alto impacto ou risco elevado, como alterações estruturais de infraestrutura, mudanças de esquema de banco de dados em produção, ou exceções ao fluxo padrão (ex.: hotfix emergencial). Em situações rotineiras, a aprovação por pares (revisão de PR) substitui o comitê formal.

*Neste projeto individual, o papel do CCM é exercido pelo próprio estudante (João Filipi Girardi Tissi), com validação do professor responsável quando aplicável. É acionado apenas em mudanças estruturais relevantes (ex.: reescrita da lógica principal do aplicativo ou mudança de escopo da atividade); mudanças de rotina seguem o fluxo padrão de commit/PR descrito na seção 8.1.*

## 9. Baselines e Releases

### 9.1 Geração Automática de Baseline

Diferente de modelos tradicionais, em que a baseline é criada manualmente em marcos do projeto, neste plano cada implantação bem-sucedida em produção gera automaticamente uma baseline: uma tag de versão (SemVer) associada ao commit, à imagem publicada no registro e ao estado da infraestrutura correspondente.

### 9.2 Notas de Versão (Release Notes)

As notas de release são geradas automaticamente pelo pipeline a partir dos Conventional Commits desde a última tag, publicadas junto à release no repositório.

## 10. Estimativa do Status de Configuração

### 10.1 Monitoramento e Observabilidade

Os ambientes de Homologação e Produção devem ser monitorados continuamente (métricas, logs e alertas), permitindo identificar rapidamente falhas decorrentes de uma implantação.

### 10.2 Métricas DORA

| Métrica | Descrição |
|---|---|
| Deployment Frequency | Frequência com que o código é implantado em produção. |
| Lead Time for Changes | Tempo entre o commit e a implantação em produção. |
| Change Failure Rate | Percentual de implantações que resultam em falha em produção. |
| Mean Time to Restore (MTTR) | Tempo médio para restaurar o serviço após uma falha. |

### 10.3 Relatórios e Auditorias

O histórico de pipelines (builds, testes, deploys) e os logs do sistema de versionamento constituem, por si, o principal insumo de auditoria de GCS, dispensando relatórios manuais na maior parte dos casos.

## 11. Marcos

*Marco 1 (concluído): criação do repositório Git e publicação da primeira versão funcional do aplicativo. Marco 2 (planejado): implementação de testes automatizados básicos (unitários) para a lógica de navegação e tradução. Marco 3 (planejado): configuração de um pipeline de CI via GitHub Actions para rodar "flutter analyze" e os testes a cada Pull Request. Marco 4 (planejado): publicação da primeira Release formal (build web e/ou APK) com tag de versão SemVer.*

## 12. Treinamento e Recursos

*Como se trata de um projeto individual e acadêmico, o principal recurso necessário é o próprio estudante, com apoio do material da disciplina de Desenvolvimento de Dispositivos Móveis (onde o aplicativo foi originalmente desenvolvido) e desta disciplina de DevOps. Ferramentas necessárias: Git, conta no GitHub, Flutter SDK e um editor de código (Android Studio ou VS Code). Para as etapas planejadas (CI via GitHub Actions), será necessário estudo introdutório sobre a sintaxe de workflows do GitHub Actions.*

## 13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Bibliotecas e serviços de terceiros integrados ao produto também são itens de configuração e devem ser versionados explicitamente (arquivo de dependências travado — lockfile).

- Análise automática de vulnerabilidades em dependências (SCA) integrada ao pipeline de CI.
- Atualizações de dependências propostas automaticamente (ex.: Dependabot/Renovate) e submetidas ao mesmo fluxo de PR.

*Não aplicável: não há subcontratados ou fornecedores externos envolvidos neste projeto individual. As únicas dependências de terceiros são os pacotes do SDK Flutter/Dart, gerenciados via pubspec.yaml.*

## 14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

| Etapa | Descrição | Gatilho |
|---|---|---|
| CI | Build, lint, testes unitários/integração, análise de segurança, publicação do artefato/imagem. | Abertura/atualização de Pull Request |
| Merge | Integração da mudança na branch principal. | Aprovação de PR + CI verde |
| CD — Staging | Deploy automático em homologação + testes de aceitação. | Merge na branch principal |

| Etapa | Descrição | Gatilho |
|---|---|---|
| Gate de Release | Aprovação para produção (automática ou manual, conforme criticidade). | Sucesso dos testes em staging |
| CD — Produção | Deploy (rolling/blue-green/canary), smoke tests, geração de baseline/tag. | Aprovação do gate de release |

***Nota:*** *este template deve ser adaptado ao contexto real do projeto (ferramentas, criticidade do sistema e maturidade da equipe em DevOps), mantendo os quatro pilares da GCS — identificação, controle de versão, controle de mudanças e auditoria/status — agora suportados por automação.*